

人工智能数学基础实验

刘绍辉

shliu@hit.edu.cn; 微信: shliu13503627854, QQ: 953444135

2026年春季学期

实验基本要求

- 实验上课时间是
 - (2-4-6-8周) 周一下午7-8节, 成教楼402, 周四下午7-8节, 成教楼402;
 - (2-4-6-8周) 周二下午7-8节, 格物213, 周四上午3-4节, 格物213.
- 实验一共四周, 八次
- 3次具体实验, 1次大作业实验
- 实验 (30%)
 - 1.线性方程组求解: 最小二乘, 正则化, RANSAC模型
 - 2.PCA+RPCA: 噪声, 主成分, 近似
 - 3.优化算法的实现: 梯度类方法+ADMM方法的实践
 - 4.大项目: 构建AI仿真器, 或者其余相关的大作业都可以! 例如: 随机模拟: 构建哈工大校园内, 学生的生活轨迹, 以及校园内个位置任意时刻的人群密度
- 分组选择主题进行讨论和讲解: 交流与沟通 (20%)

实验基本要求

- 编程语言和编程平台不定
 - 大家可以任意选择自己熟悉的编程语言和编程平台
 - Mworks科教版2026a（苏州同元）可以替换Matlab（哈工大禁止使用matlab）
 - Julia, python等语言都可以，使用anaconda+jupyter开发即可，Julia有JuMP优化接口，并能调用商业求解器，python也有很多numpy等包供调用。
 - Opencv+vscode(后者visual studio C++, 2019等都可以)
 - 可以使用先用的库和开源代码，但需要把算法的内容深入理解清楚，可以百度搜索、百度百科、知乎、博客等都可以
 - 需要自己写代码，或者将代码有机的组成一个能解决实验问题的方案
- 交流与沟通
 - 阅读报告、文档
 - PPT+讲解

Julia资料

- 1.北大李东风写的 Julia语言入门 教材:
https://www.math.pku.edu.cn/teachers/lidf/docs/Julia/html/_book/index.html
- 2.对科学计算来说, Julia多重分派意味着什么? :
<https://zhuanlan.zhihu.com/p/569704329>
- 3.陈久宁在华师大做的 Julia 小课堂 的一些学习材料:
https://github.com/johnnynchen94/Julia_and_its_applications
- 4.快速入门: 一份简单而粗略的语言概览 :
<https://cheatsheet.juliadocs.org/zh-cn/>

Julia优化接口JuMP: <https://jump.dev/>

- 调用JuMP接口

```
max    3x1 - x2 - x3
Subject to  - 2x1 + x3 = 1
            - 4x1 + x2 + 2x3 ≥ 3
            x1 - 5x2 + x3 ≤ 11
            x1 ≥ 0
            x2 ≥ 0
            x3 ≥ 0
```

```
Optimal Soutions:
x1 = 1.6666666666666667
x2 = 1.0
x3 = 3.3333333333333335
```

```
using JuMP, GLPK
m = Model(GLPK.Optimizer)
@variable(m, 0 <= x1)
@variable(m, x2 >= 0)
@variable(m, x3 >= 0)
@objective(m, Max, 3x1 - x2 - x3)
@constraint(m, constraint1, x1 - 5x2 + x3 <= 11)
@constraint(m, constraint2, -4x1 + x2 + 2x3 >= 3)
@constraint(m, constraint3, -2x1 + x3 == 1)
print(m)

JuMP.optimize!(m)
println("Optimal Soutions:")
println("x1 = ", JuMP.value(x1))
println("x2 = ", JuMP.value(x2))
println("x3 = ", JuMP.value(x3))
```

关于数学变换的实验内容

- 采用平台阅读一幅图像，如果是彩色图像，转化为灰度图像；实现图像按照指定块划分，并置乱块的位置显示图像，然后尝试恢复置乱后的图像，能获取图像任意一点的像素值，能将图像分成任意块大小（可以将块的长可宽设置成参数，不足的补0或255），
`PermutationFun(inputImage, blockwidth, blockheight, seed)`, 例如4*4, 8*8, 16*16, 32*32, 64*64, 并置乱块的位置并显示（类似马赛克效果）；能指定区域内的图像分块并置乱块的顺序再显示（可以调用软件或库的读图接口）
 - 编程实现二维8*8图像块的DFT变换及其逆变换
 - 输入一幅RGB图像，将其转化为YCbCr颜色空间，然后对Y分量做分块8*8的二维DFT变换，保存为灰度图像，统计计算时间，显示幅度谱、相位谱、能量谱
 - 分别只保留幅度或相位，做逆变换，显示图像
 - 编程实现二维8*8图像块的DCT变换及其逆变换
 - 编程实现8*8块DCT变换，并思考如何用FFT来实现DCT变换
 - 显示2维8*8DCT变换的基函数图像
 - 理解DCT变换的能量聚集特性：当使用8*8的二维DCT变换系数矩阵时，若只保留8*8变换左上角的一些系数值，其余系数值都设置为0，然后进行2维DCT逆变换，请比较保留1, 2, 4, 6, 8, 10个系数时，其重构图像与原始图像的PSNR和SSIM值.(将保留系数值的个数按照zig-zag扫描方式作为函数参数)

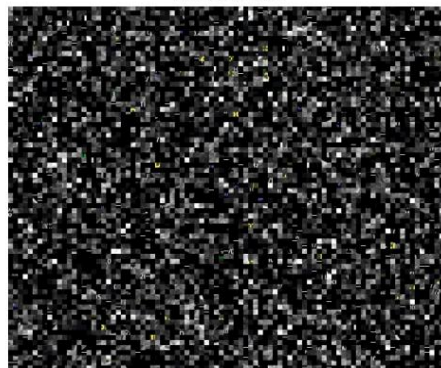
Y (YCbCr)



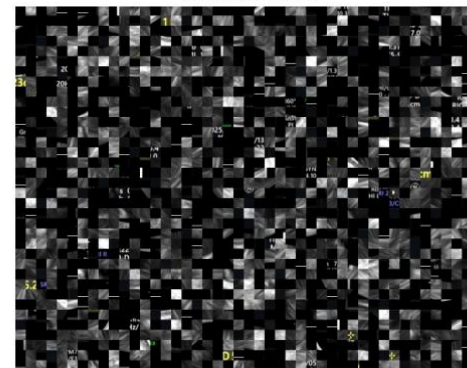
Permuted 4x4



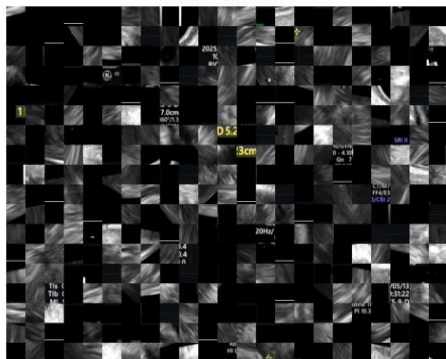
Permuted 8x8



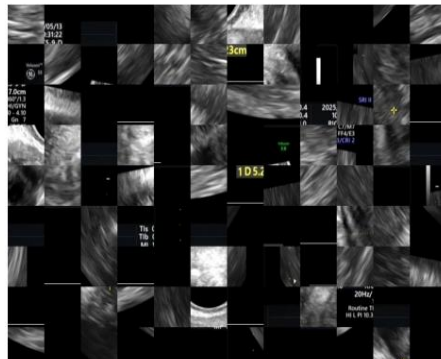
Permuted 16x16



Permuted 32x32



Permuted 64x64



Region Permuted (center 256x256)



$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

DCT

- 最常见的DCT变换为DCTII型变换

- $X(k) = \sum_{n=0}^{N-1} x(n) \cos \left[\frac{\pi(2n+1)k}{2N} \right], k = 0, \dots, N-1$

- 其边界条件对应为: $x(n)$ 关于 $n = -\frac{1}{2}$ 偶对称, $n = N - \frac{1}{2}$ 偶对称; $X(k)$ 在 $k = 0$ 偶对称, $k = N$ 奇对称

- 一般情况

- $k = 0, X(k) = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x(n)$

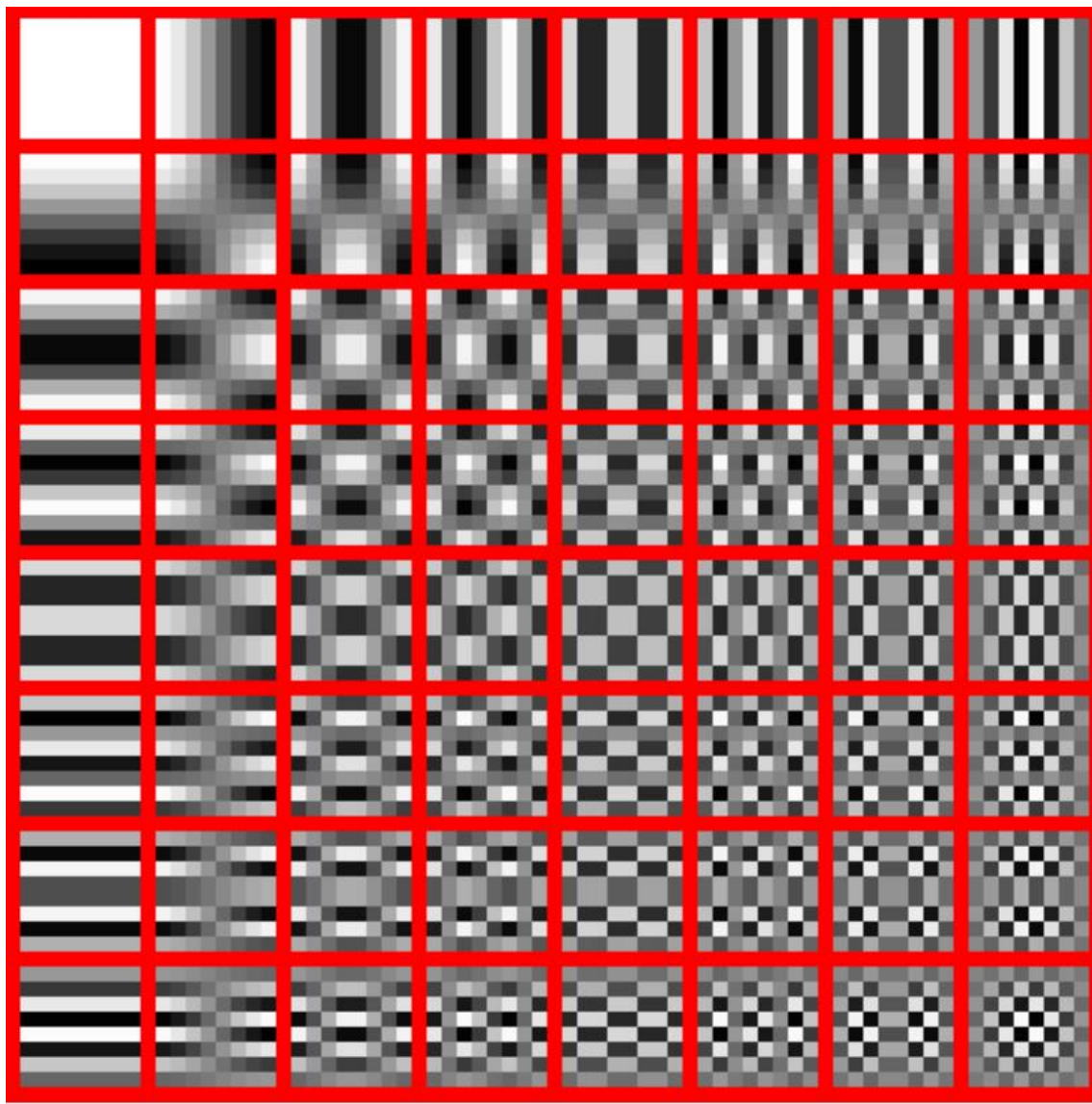
- $k \in [1, N-1], X(k) = \sqrt{\frac{2}{N}} \sum_{n=0}^{N-1} x(n) \cos \left[\frac{(2n+1)k\pi}{2N} \right]$

- $x(n) = \frac{1}{\sqrt{N}} X(0) + \sqrt{\frac{2}{N}} \sum_{k=1}^{N-1} X(k) \cos \left[\frac{(2n+1)k\pi}{2N} \right], n \in [0, N-1]$

- 矩阵形式

- $\mathbf{X} = \mathbf{C}\mathbf{x}, \mathbf{C} = [c(k, n)], c(k, n) = \begin{cases} \frac{1}{\sqrt{N}}, k = 0, n \in [0, N-1] \\ \sqrt{2/N} \cos \left[\frac{(2n+1)k\pi}{2N} \right], k \in [1, N-1], n \in [0, N-1] \end{cases}, \text{且 } \mathbf{x} = \mathbf{C}^{-1}\mathbf{X} = \mathbf{C}^t\mathbf{X}, \mathbf{C} = \mathbf{C}^*, \mathbf{C}^{-1} = \mathbf{C}^t$

64个8x8的2维DCT基函数



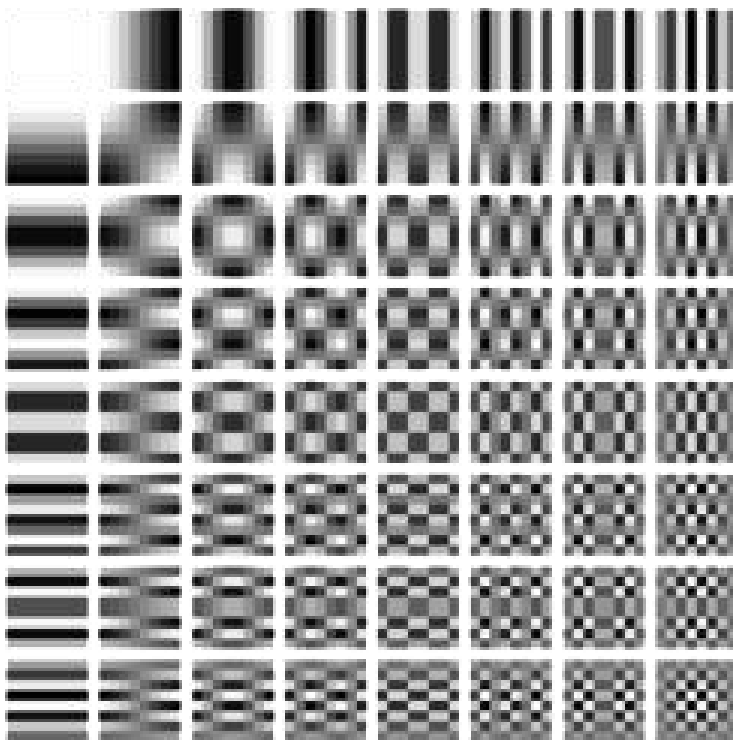
DCT(Discrete)变换的例子

- 假设图像大小为 8×8



原始大小图像, 扩大10倍(最近邻方法), 扩大10倍(双线性差值).

- DCT基函数



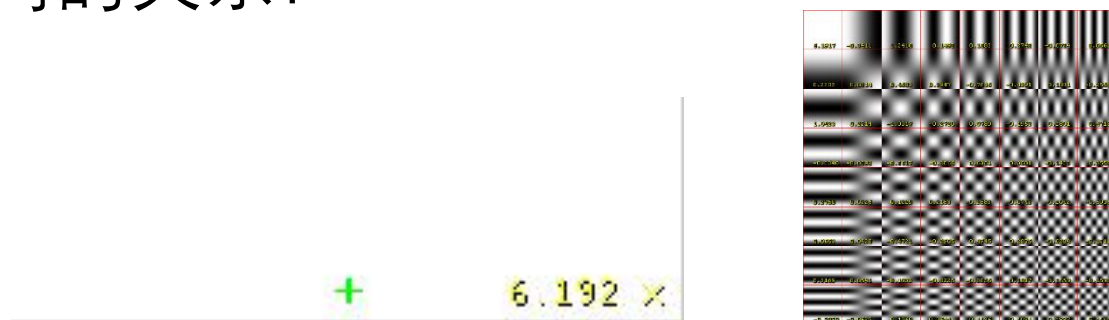
0.1917	-0.3411	1.2418	0.1492	0.1583	0.2742	-0.0724	0.0561	
0.2205	0.0214	0.4503	0.3947	-0.7846	-0.4391	0.1001	-0.2554	
1.0423	0.2214	-1.0017	-0.2720	0.0789	-0.1952	0.2801	0.4713	
-0.2340	-0.0392	-0.2617	-0.2866	0.6351	0.3501	-0.1433	0.3550	
0.2750	0.0225	0.1229	0.2183	-0.2583	-0.0742	-0.2042	-0.5906	
0.0653	0.0423	-0.4721	-0.2905	0.4745	0.2875	-0.0284	-0.1311	
0.3169	0.0541	-0.1033	-0.0225	-0.0055	0.1017	-0.1650	-0.1500	
-0.2970	-0.0627	0.1960	0.0644	-0.1136	-0.1031	0.1837	0.1444	

DCT变换的例子

• DCT系数

6.1917	-0.3411	1.2418	0.1492	0.1583	0.2742	-0.0724	0.0561
0.2205	0.0214	0.4503	0.3947	-0.7846	-0.4391	0.1001	-0.2554
1.0423	0.2214	-1.0017	-0.2720	0.0789	-0.1952	0.2801	0.4713
-0.2340	-0.0392	-0.2617	-0.2866	0.6351	0.3501	-0.1433	0.3550
0.2750	0.0226	0.1229	0.2183	-0.2583	-0.0742	-0.2042	-0.5906
0.0653	0.0428	-0.4721	-0.2905	0.4745	0.2875	-0.0284	-0.1311
0.3169	0.0541	-0.1033	-0.0225	-0.0056	0.1017	-0.1650	-0.1500
-0.2970	-0.0627	0.1960	0.0644	-0.1136	-0.1031	0.1887	0.1444

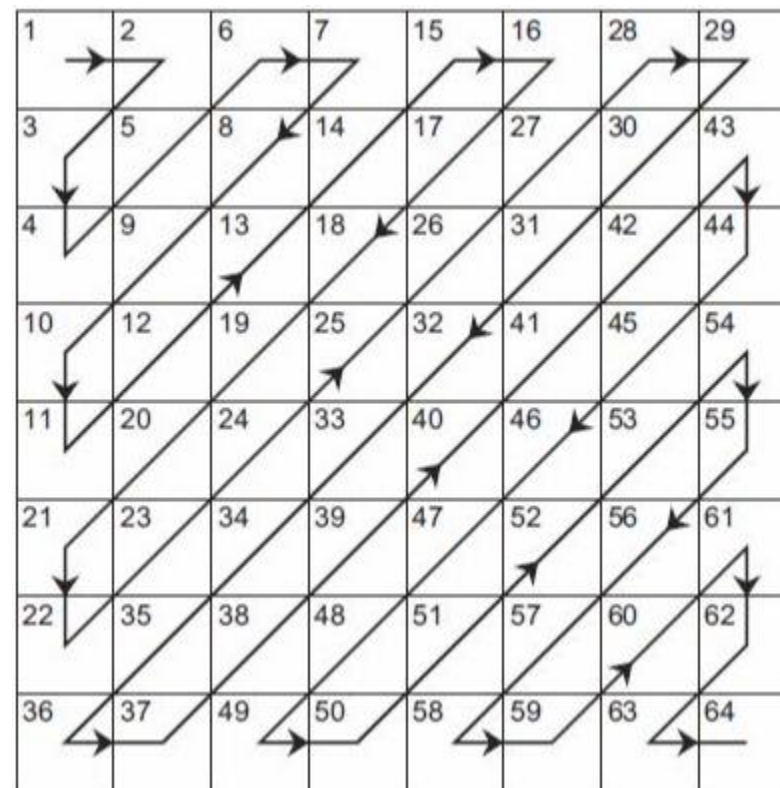
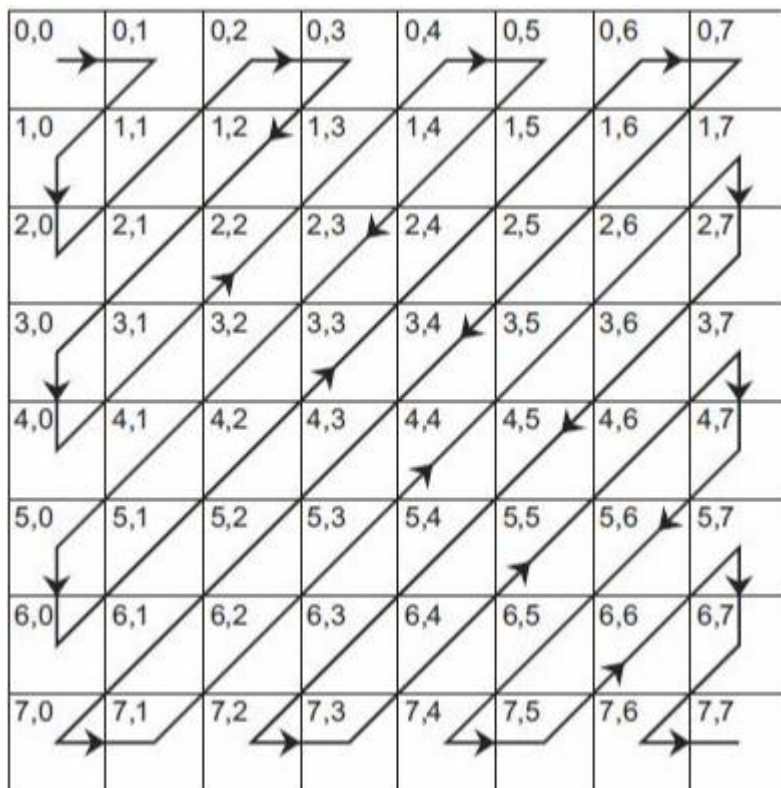
• 基函数和变换系数之间的关系?



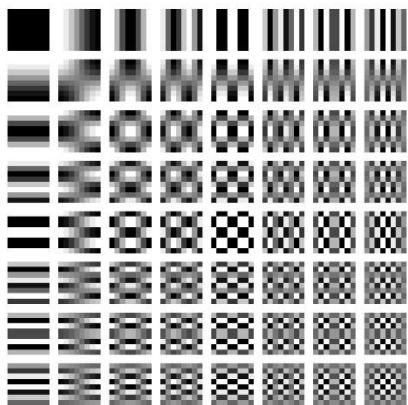
左边为最后图像. 中间是加权函数 (基×系数) 加入到左边图像后更新显示. 右边是当前函数和相应的系数. 图像使用双线性差值扩大10倍后用于显示.

DCT变换

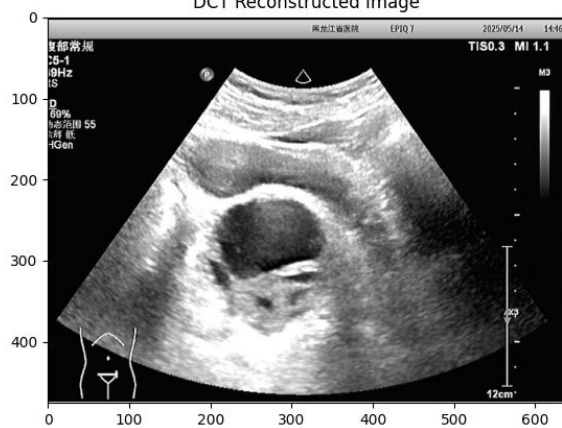
- 8×8 分块变换
 - Zigzag扫描(zigzag scan)



8x8 DCT Basis



DCT Reconstructed Image



Reconstruction with 1 coefficients



Reconstruction with 2 coefficients



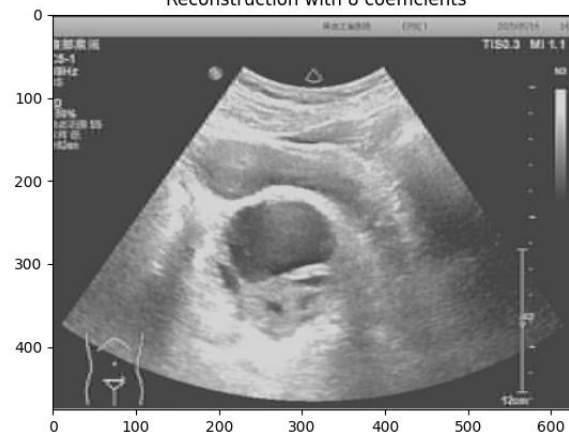
Reconstruction with 4 coefficients



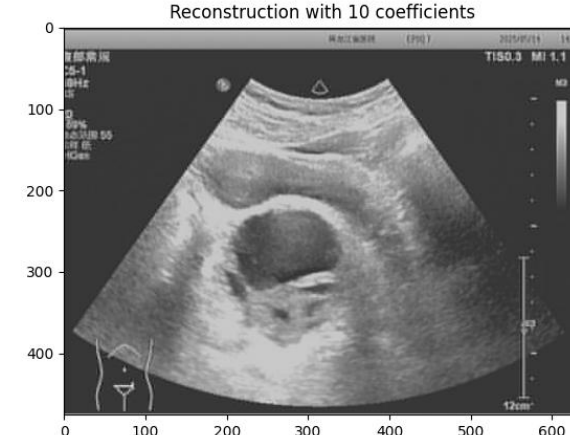
Reconstruction with 6 coefficients



Reconstruction with 8 coefficients



Reconstruction with 10 coefficients



实验一：数据拟合-基本内容

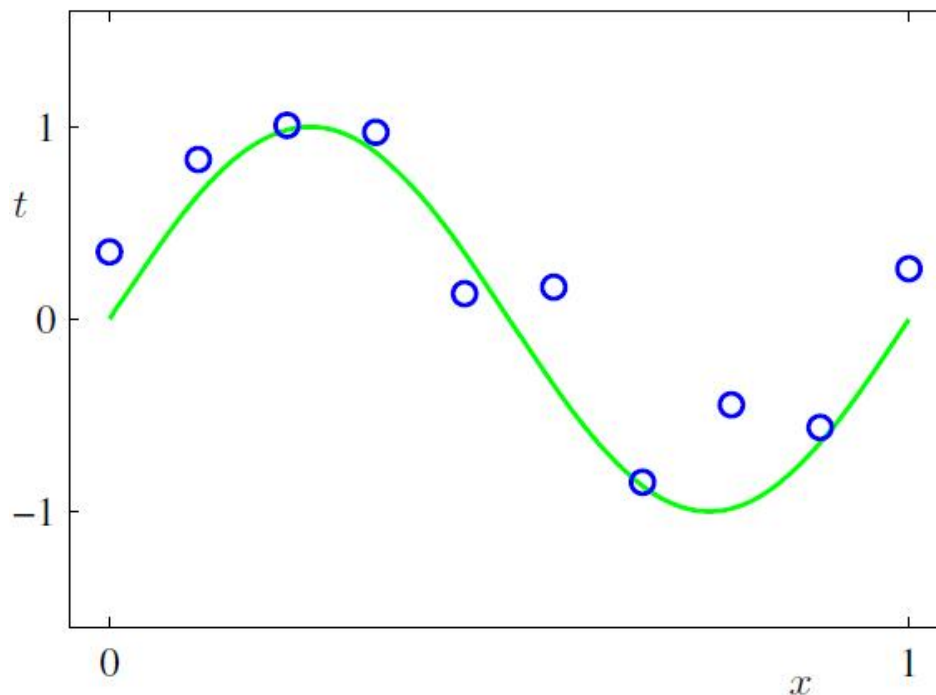
- 给定一些点，如何来进行直线拟合？（ransac方法和最小二乘方法）
 - 如果是根据直线方程+噪声，生成的一些点，如何来拟合直线？如果有很多外点(outlier)如何处理？
 - 如果是从图像中检测出来的边缘点，这些边缘点正好在直线上，如何来进行直线拟合？
 - 如果这些离散点形成了平行直线，如何来拟合出平行直线的方向？
- 具体内容（分别使用ransac方法和最小二乘方法来进行拟合）
 - 1.直线拟合
 - 根据直线方程 $ax+by+c=0$,产生 (x_i, y_i) 随机点，然后增加随机噪声成为: $(x_i + n_i, y_i + m_i)$,根据这些点，拟合直线 $\mathbf{ax + by + c = 0}$ 中的参数
 - 如果有一系列平行直线 $\mathbf{ax + by + c_1 = 0, ax + by + c_2 = 0, ax + by + c_3 = 0}$,然后对直线上的点添加类似的噪声，拟合这些平行直线
 - 2.曲线拟合
 - 自己设计曲线方程，例如圆方程，椭圆方程，然后添加适当的噪声（例如高斯噪声），然后分别采用ransac和最小二乘方法进行拟合
 - 如果添加一些外点，拟合效果如何？是否有方法改进！

自定义产生数据对的函数

- l_1 范数是指向量元素的绝对值的和; l_2 范数是指向量元素的平方值的和。
- 假设给定函数 $y = a\sin\omega_1 t + b\sin\omega_2 t$ (可以自定义各种函数都可以), 从中生成 n 个点 $(t_i, \hat{y}_i), i = 1, 2, \dots$, 假设点 $\hat{y}_i = y_i + n_i$, $y_i = a\sin\omega_1 t_i + b\sin\omega_2 t_i$, 其中 n_i 是服从高斯分布或者拉普拉斯分布的随机噪声, 然后用**拟合、回归和神经网络**的方法来求解模型 f , 使得模型 $f(t, w)$ 拟合原函数的误差最小!

最小二乘数据拟合仿真

- 10个训练数据点：圆圈代表输入变量 x 的观察值， t 代表对应的目标值，绿色线表示 $\sin(2\pi x)$ ，用来生成数据
- 问题：如何预测任意输入变量 x 所对应的输出值 t ? 注意, 绿色线未知



最小二乘数据拟合仿真

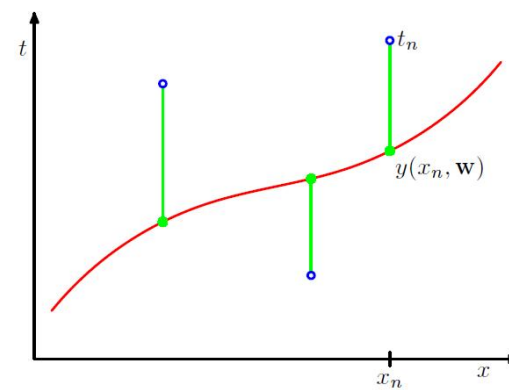
- 简单采用多项式来进行逼近

$$y(x, \mathbf{w}) = w_0 + w_1x + \cdots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

- 未知量与目标，函数之间为线性关系，称为线性模型，如何来确定系数 w_i 呢？

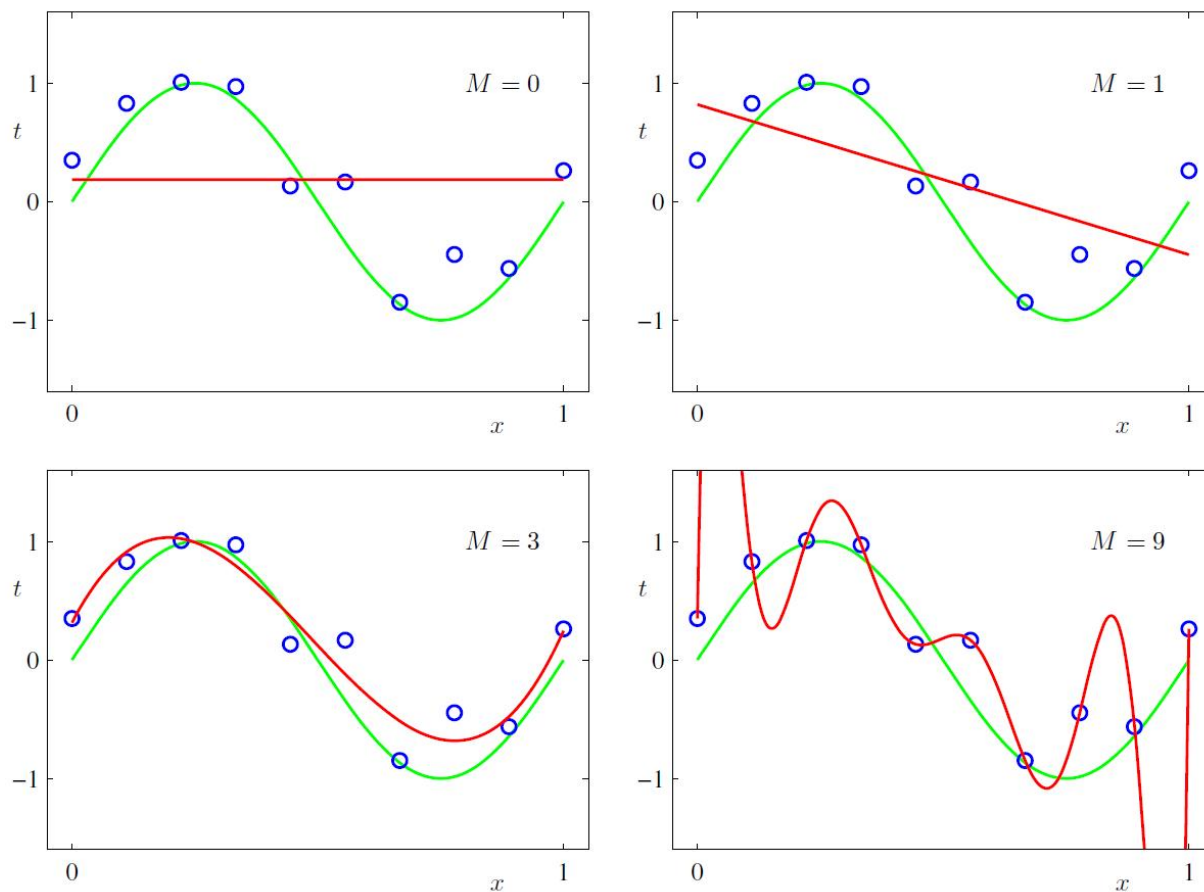
- 通过训练数据，如何做？
- 形式化为一个最优化问题

$$\min E(\mathbf{w}), E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2$$



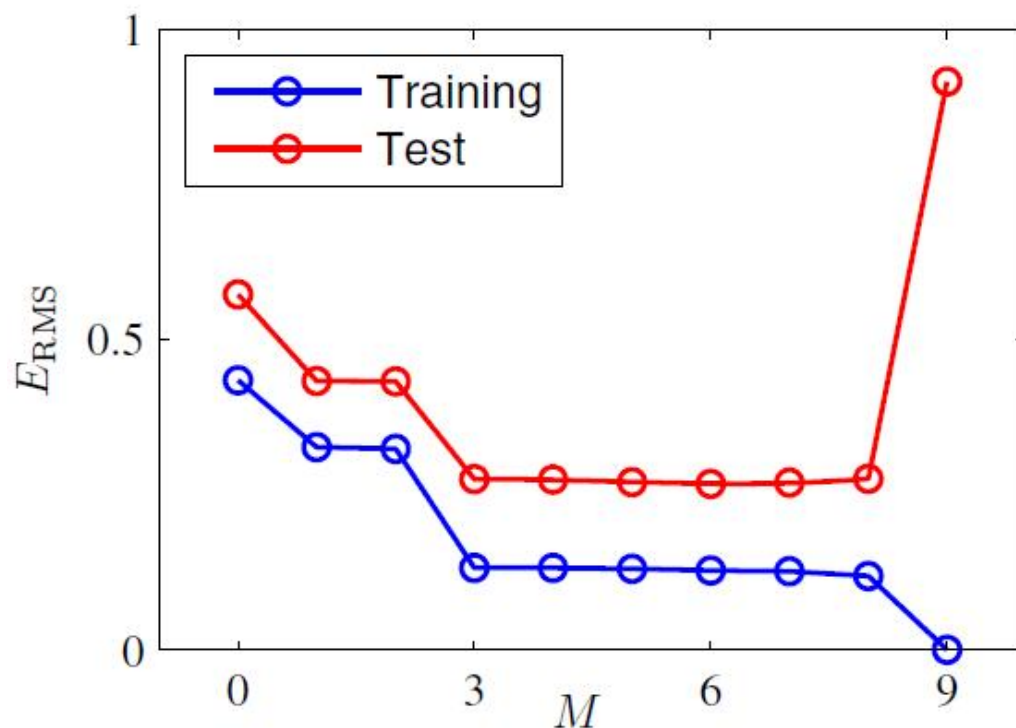
最小二乘数据拟合仿真

- 如何求解？不同的 M 有什么影响？



最小二乘数据拟合仿真

- 采用 *Root – Mean – Squre(RMS)* 误差评价: $E_{RMS} = \sqrt{2E(w^*)/N}$



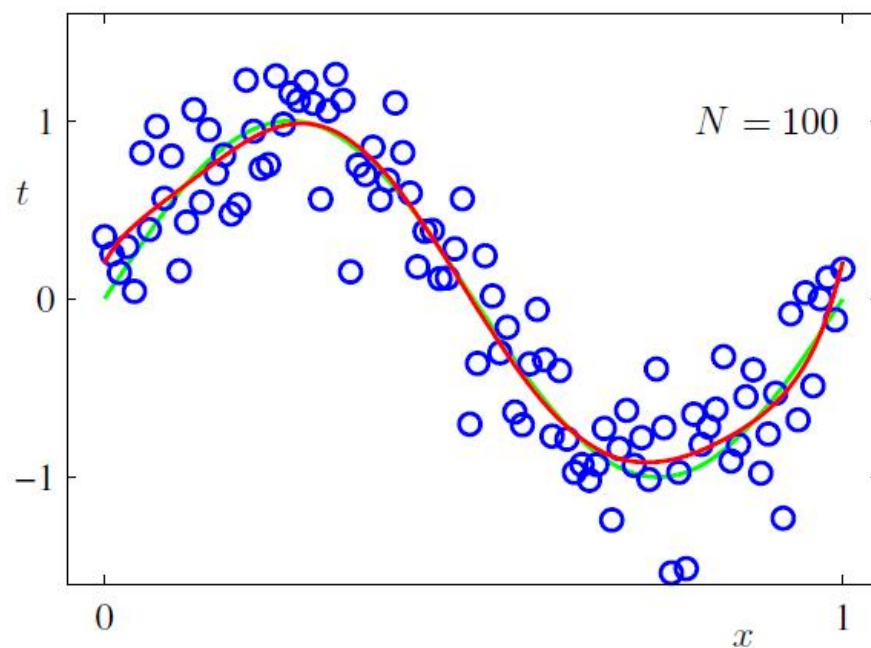
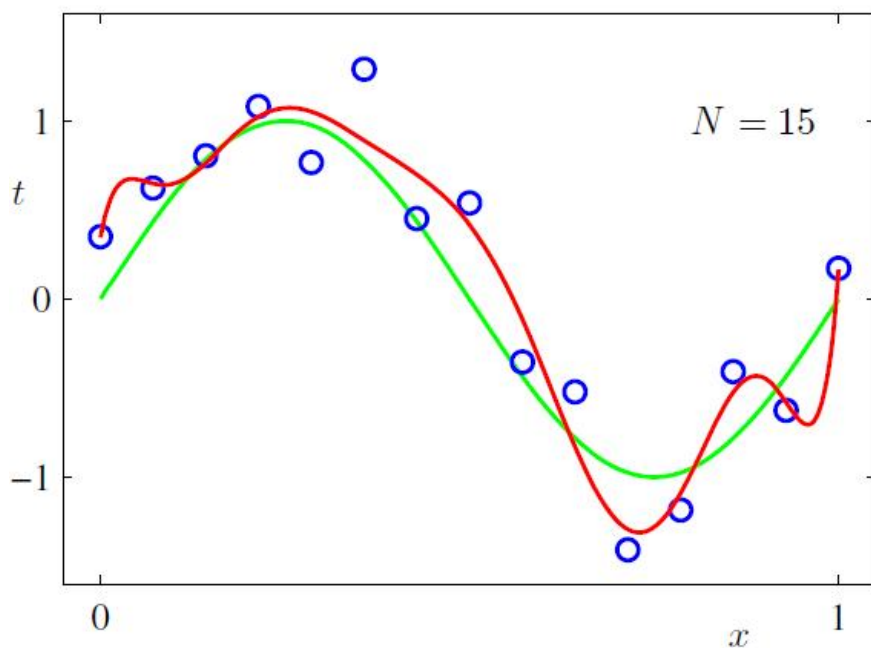
最小二乘数据拟合仿真

- 权系数的情况

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

最小二乘数据拟合仿真

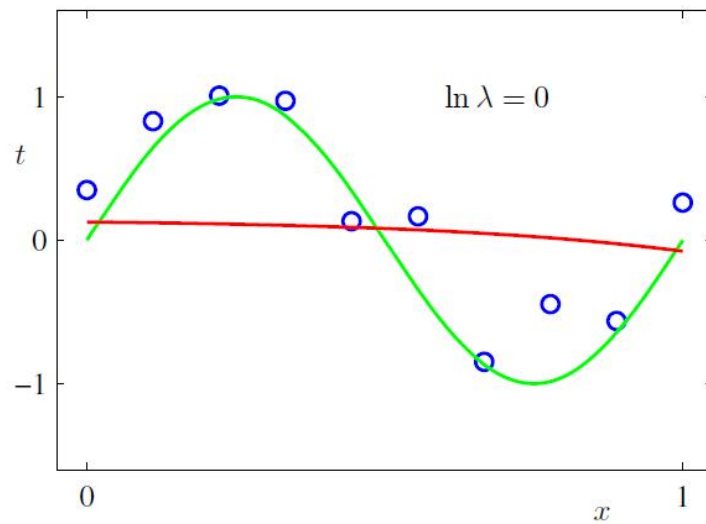
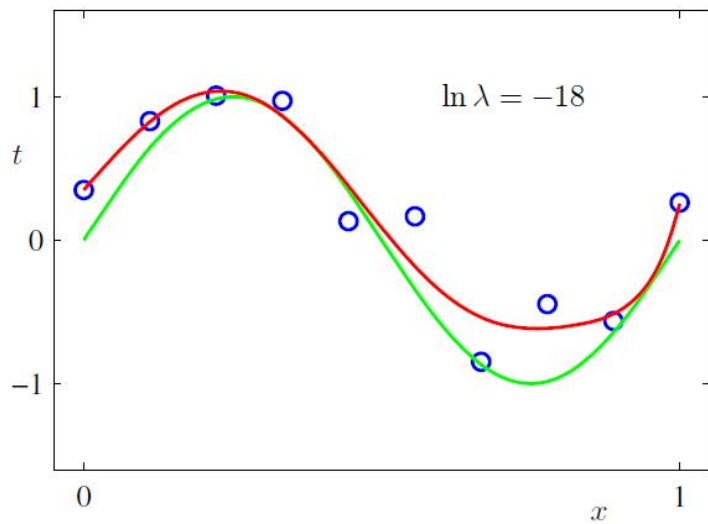
- 提升训练数据量($M = 9$)



- 一般数据大小为参数的5-10倍左右, 那是否需要限制模型参数的数量呢?

最小二乘数据拟合仿真

- 如何改进？正则化方法(Regularization)
- 对权系数进行限定，对近似模型进行约束
 - $\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$
 - 也请一并尝试： $\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \mathbf{w}) - t_n\}^2 + \lambda \|\mathbf{w}\|_1$
- 系数 λ 控制正则项与误差项之间的均衡程度



最小二乘数据拟合仿真

- 正则因子对权系数的影响

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

最小二乘数据拟合仿真

- 没有外点的情况下：
 - 1.请完成上述实验过程，包括 l_2 和 l_1 范数，并提供实验对比分析；
 - 2.针对上述情况，请完成一个简单的MLP（多层感知机）来拟合给定的方程，并进行对比分析；
- 有外点的情况下：
 - 1.请完成上述不同方法拟合性能的对比分析
 - 2.探讨还有什么好方法来进行拟合？
 - 3.请查阅文献，了解滑动平均等思想，并设计一个比最小二乘更好的方案来处理存在外点的情况！

基本内容

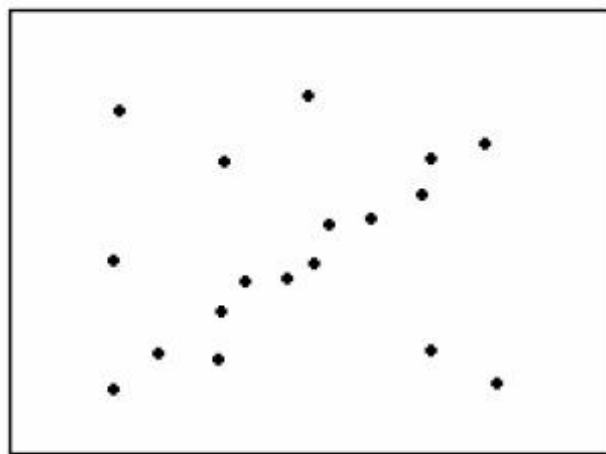
- 可进一步应用到单应矩阵、基本位姿估计、三维点云匹配当中（参考下面的文献!）（可选）
- https://docs.opencv.org/4.5.0/d1/df1/md_build_master-contrib_docs%20%20lin64_opencv_doc_tutorials_calib3d_usac.html
- Opencv 4.5.0中已经集成了一些相关的RANSAC方法，有些方法在CVPR Image Matching Challenges中进行了测试，取得了很好的结果。
- [Image Matching across Wide Baselines: From Paper to Practice](#)根据这篇论文的结果，Marginalizing Sample Consensus（MAGSAC++）取得了最好的结果！该文章发表在2022年的TPAMI上，因此以后大家有要用到的时候，请以MAGSAC++为基准。
- 实验中可以使用：<https://github.com/danini/magsac>

RANSAC

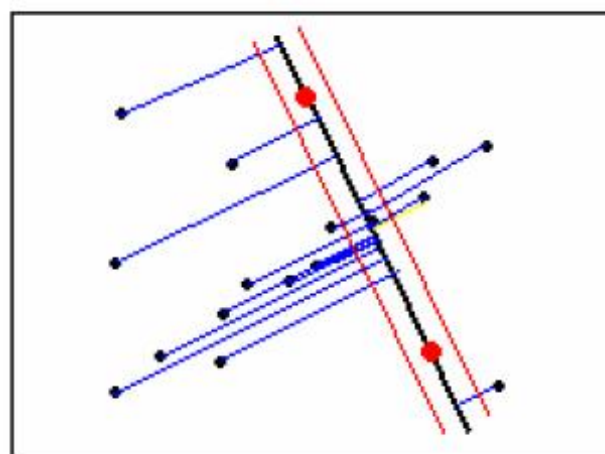
- 基本思想
 - 随机选择两点(确定一条直线所需要的最小点集); 由这两个点确定一条线 l ;
 - 根据阈值 t , 确定与直线 l 的几何距离小于 t 的数据点集 $S(l)$, 并称它为直线 l 的一致集;
 - 重复若干次随机选择, 得到直线 $l_1, l_2, \dots, l_n$ 和相应的一致集 $S(l_1), S(l_2), \dots, S(l_n)$;
 - 使用几何距离, 求最大一致集的最佳拟合直线, 作为数据点的最佳匹配直线

RANSAC

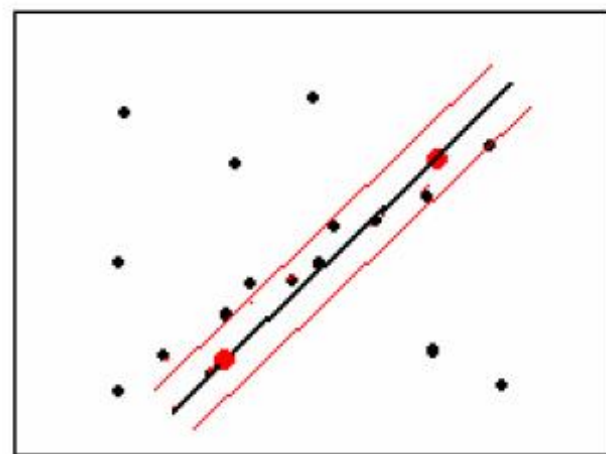
- 基本思想



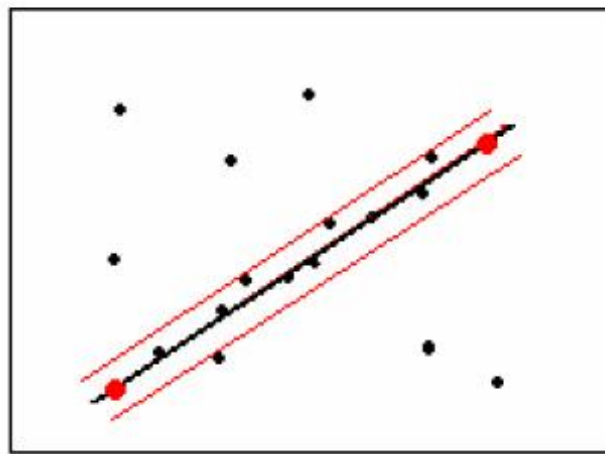
(a)



(b)



(c)



(d)

直线的 RANSAC 估计：(a)是来自一条直线的 18 个测量数据点，在给定的距离阈值下有 7 个外点；(b)是两个外点所确定的直线，它的一致集仅含有三个数据点；(c)是由一个内点与一个外点所确定的直线，它的一致集含有 8 个数据点；(d)是 RANSAC 最终估计的直线，它具有最大的一致集，并且所有的外点都被去除。

RANSAC

- 推广到一般的模型，估计模型参数 p
 - 确定求解模型 M ，即确定模型参数 p ，所需要的最小数据点的个数 n 。由 n 个数据点组成的子集称为模型 M 的一个样本；
 - 从数据点集 D 中随机地抽取一个样本 J ，由该样本计算模型的一个实例 $M_p(J)$ ，确定与 $M_p(J)$ 之间几何距离 $<$ 阈值 t 的数据点所构成的集合，并记为 $S(M_p(J))$ ，称为实例 $M_p(J)$ 的一致集；
 - 如果在一致集 $S(M_p(J))$ 中数据点的个数 $\# S(M_p(J)) >$ 阈值 T ，则用 $S(M_p(J))$ 重新估计模型 M ，并输出结果；如果 $\# S(M_p(J)) <$ 阈值 T ，返回到步骤2；
 - 经过 K 次随机抽样，选择最大的一致集 $S(M_p(J))$ ，用 $S(M_p(J))$ 重新估计模型 M ，并输出结果

RANSAC

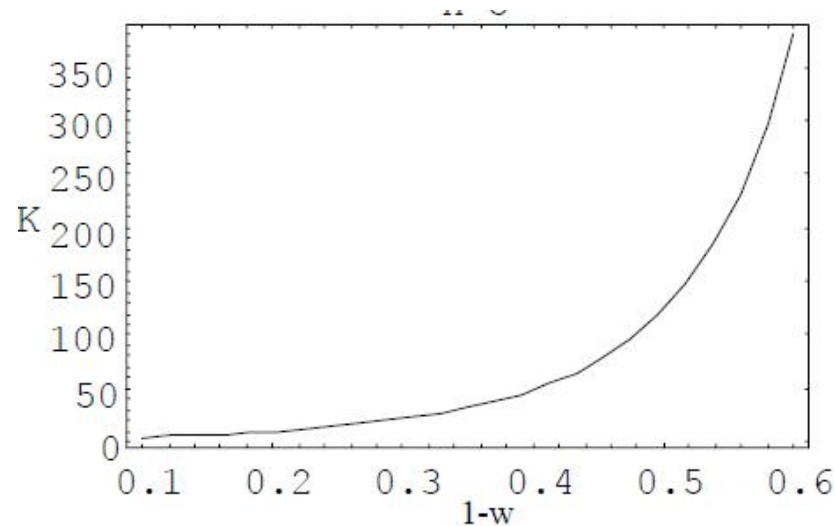
- 实现中需要注意的事项
 - 抽样次数的考虑

表：样本所含数据点个数 n 与抽样次数 K 的一些对应值 ($w = 0.45$, $z = 0.02$)

n	2	3	4	5	7	9	13	16	20
K	18	41	94	210	1045	5168	126076	1.38×10^6	3.37×10^7

- 内点比例为 w , K 次取中所有样本均为坏样本的概率，从而可得

$$K = \frac{\log z}{\log(1 - w^n)}$$



图：抽样次数 K 与外点的比例 $(1-w)$ 之间的变化关系

RANSAC

- 距离阈值 t
 - 经验选取
 - 如果测量误差服从0均值，sigma方差的高斯分布，则可以计算 t
 - 因为这时候点到模型几何距离的平方是高斯变量的平方和，服从自由度为 n 的卡方分布

$$p_{\chi_n^2}(x) = \begin{cases} \frac{1}{2^{n/2} \Gamma(n/2)} e^{-x/2} x^{n/2-1}, & 0 < x < \infty \\ 0, & \text{其它} \end{cases}$$

RANSAC

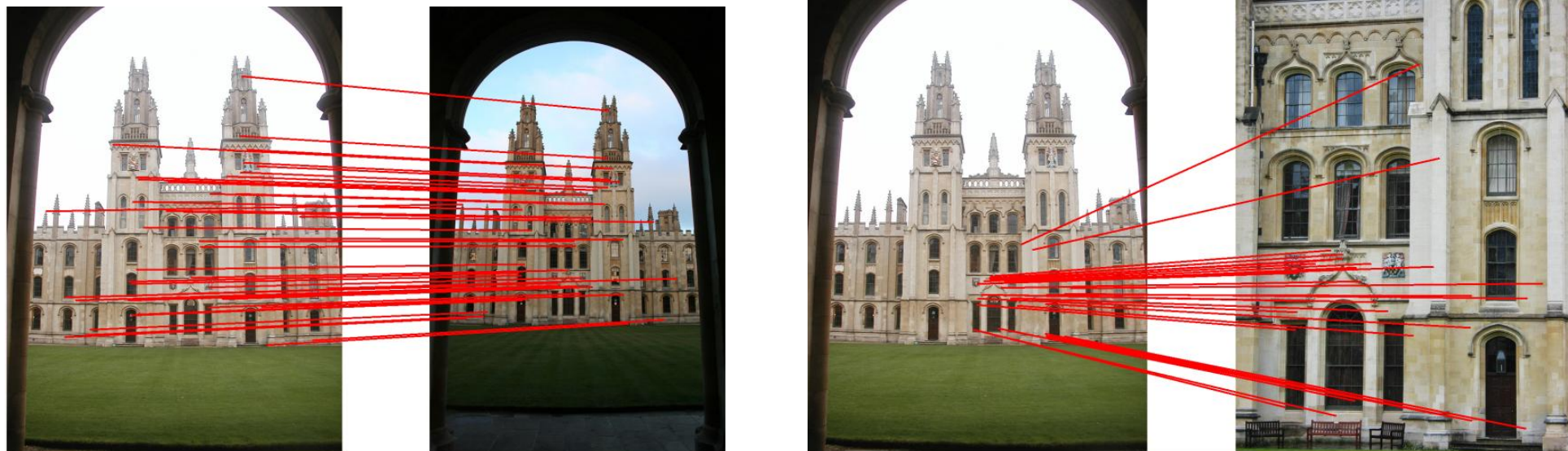
- 终止阈值
 - 不好设置
 - 一般规则：如果根据内点比例的估计值，内点数目与一致集大小相当的时候就停止
 - 自适应规则

自适应算法(终止 RANSAC 抽样):

- 1) 对内点比例作最保守估计 $w = w_0$ (如 $w_0 = 0.1$ ，这意味着在数据点集中可能有 90% 的外点，这确实是一个保守估计)，应用公式 $K = \frac{\log \varepsilon}{\log(1 - w^n)}$ 得到抽样次数 K 的初始值 K_0 ；
- 2) 抽样并更新 w_0, K_0 ：令当前抽样的一致集所含数据点占整个数据点的比例为 w ，若 $w > w_c$ 则更新 $w_0 := w$ ，并且应用公式(17.1.2)更新抽样数 K_0 ；否则，保持原来的 w_0, K_0 ；
- 3) 如果抽样次数已达到或超过 K_0 ，则终止抽样；否则，返回步骤 2)。

RANSAC

- 实践证明非常有效！



[1] R. Szeliski. Image alignment and stitching: A tutorial. Foundations and Trends in Computer Graphics and Vision, 2(1):1 – 104, 2006.

实验二： PCA+RPCA

- 主成份分析和鲁棒主成份分析
 - 理解基本原理
 - **用PCA和RPCA用来对mnist数据集进行分类**（可以用任何数据集的处理）
- PCA
 - <https://github.com/topics/principal-component-analysis-pca>
- Robust-PCA
 - <https://github.com/dlaptev/RobustPCA>
 - **Learned Robust PCA: A Scalable Deep Unfolding Approach for High-Dimensional Outlier Detection**
- 可参考网上代码，例如
https://benmilanko.com/projects/mnist_with_pca/

实验三：梯度下降法、牛顿法，以及Lion 基于学习的加速方法的实现与实验

- 掌握基本的优化思想及算法
- 掌握加速的思想
- 如何使用这些基本的方法来求解实际问题
- 逼近的思想的体现
- 一阶近似： $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$
- 期望新的值比 $f(x_0)$ 更小，越小越好，令 $x - x_0 = \lambda_k p_k$ ，则得：
- $f(x) = f(x_0) + \lambda_k f'(x_0) \cdot p_k$ ，注意后面为两个向量乘积，可以表示为模长乘向量夹角的表示：
 $f(x) = f(x_0) + \lambda_k \cdot \|f'(x_0)\| \cdot \|p_k\| \cdot \cos(\alpha)$ ，其中夹角 α 表示向量 $f'(x_0)$ 与 p_k 的夹角！
- 显然 $\cos(\alpha) = -1$ 时最小，表明这两个向量方向相反，因此有：
- $p_k = -\frac{f'(x_0)}{\|f'(x_0)\|}$ ，因此就有：

$$x_{new} = x_k + \lambda_k p_k = x_k - \frac{\lambda_k f'(x_0)}{\|f'(x_0)\|} = x_k - \frac{\lambda_k}{\|f'(x_0)\|} f'(x_0)$$

- 因此， x_{new} 是一个比 x_k 更好的近似

梯度下降求解二项逻辑回归模型

- 二项逻辑回归模型的条件概率分布如下
- $P(X = 1|Z) = \frac{1}{1+e^{-\theta^T z}}; P(X = 0|Z) = 1 - \frac{1}{1+e^{(-\theta)^T x}}$
- $z \in R^n$ 是输入, $X \in \{0, 1\}$ 是输出, θ^T 是参数, 应用时, 根据两个条件概率值的大小将实例分到概率值较大的一类
- 问题, 如何具体求解?
- $h_\theta = \frac{1}{1+e^{(-\theta)^T z}}$ 作为似然函数, 采用采用梯度下降法来进行求解

梯度下降求解二项逻辑回归模型

- $L(\theta) = \prod_{i=1}^m P(x_i|z_i; \theta) = \prod_{i=1}^m (h_{\theta}(z_i))^x (1 - h_{\theta}(z_i))^{1-x}$
- 取对数, 乘以 $\left(-\frac{1}{m}\right)$ 将求最大值问题转化为求最小值的问题
- $L(\theta) = -\frac{1}{m} \sum_{i=1}^m [x_i \log h_{\theta}(z_i) + (1 - x_i) \log(1 - h_{\theta}(z_i))]$
- 如何求解?
- $\theta_{j+1} = \theta_j - \alpha \frac{\partial L(\theta)}{\partial(\theta)} = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m z_i [h_{\theta}(z_i) - x_i]$
- 逻辑回归算法

梯度下降求解二项逻辑回归模型-逻辑回归算法

- **输入**：训练数据集 $T = \{(z_i, x_i), i = 1, \dots, N\}$, $z_i \in \mathcal{Z} \subseteq R^n, x_i \in \mathcal{X} \subseteq R^n$, 损失函数 $Cost(x, f(z))$
- **输出**：逻辑回归模型 $\hat{f}(z)$
- 1. 随机初始化参数 θ ;
- 2. 对样本 $i = 1, 2, \dots, N$,
- 计算 $\theta_{j+1} = \theta_j - \alpha \frac{\partial L(\theta)}{\partial (\theta)} = \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m z_i [h_{\theta}(z_i) - x_i]$
- 递归得到逻辑回归模型

实验三：梯度下降法、牛顿法，以及Lion 基于学习的加速方法的实现与实验

- Newton's方法：利用二次式来逼近原函数
- $q(\lambda) = f(\lambda_k) + f'(\lambda_k)(\lambda - \lambda_k) + \frac{1}{2}f''(\lambda_k)(\lambda - \lambda_k)^2$
- 令其导数为0，得 $\lambda_{k+1} = \lambda_k - \frac{f'(\lambda_k)}{f''(\lambda_k)}$
- 注意原函数二次可微，且 $f''(\lambda_k) \neq 0$
- 例： $f(\lambda) = \begin{cases} 4\lambda^3 - 3\lambda^4 & \text{若 } \lambda \geq 0 \\ 4\lambda^3 + 3\lambda^4 & \text{若 } \lambda < 0 \end{cases}$ ，令 $\lambda_1 = 0.4$ ，结果如下，若令 $\lambda_1 = 0.6$ ，则发生震荡，但若初始点离极小点足够近，则保证收敛

实验三：梯度下降法、牛顿法，以及Lion 基于学习的加速方法的实现与实验

- 梯度下降法在深度学习的训练优化算法中有广泛的应用
 - 梯度下降
 - 随机梯度下降
 - 各种加速算法
 - Lion加速算法

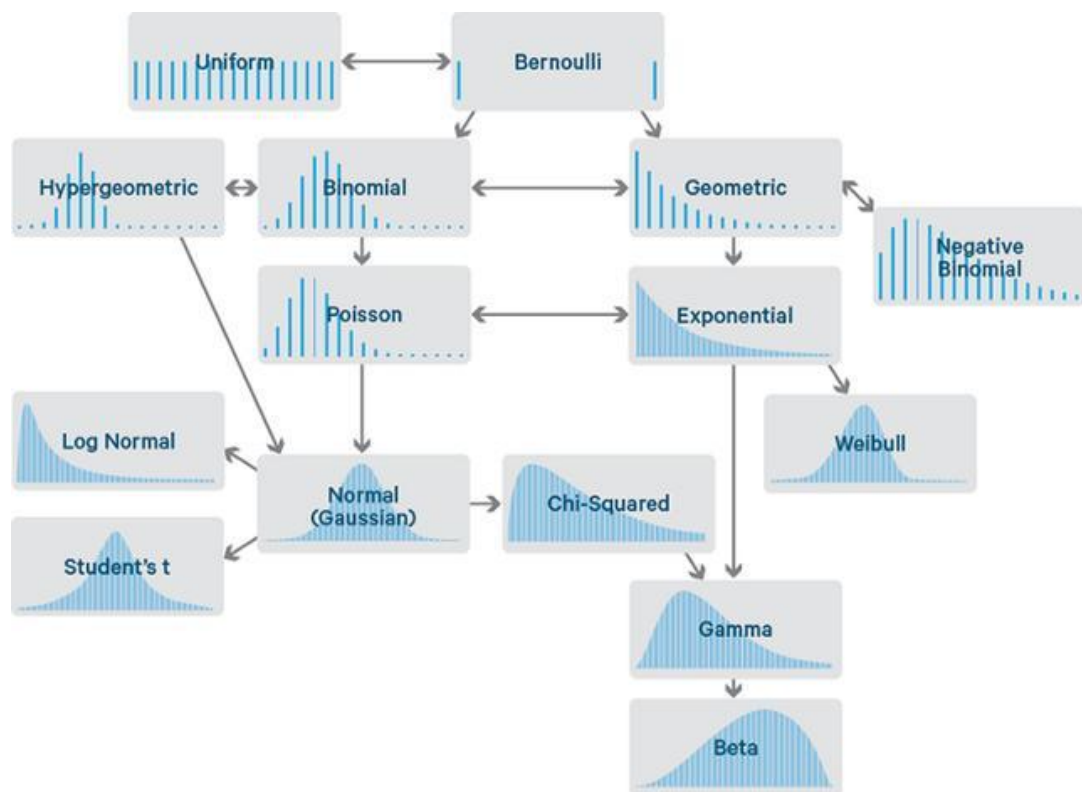
大家查阅知乎专栏：https://zhuanlan.zhihu.com/p/431298412?utm_id=0
Lion优化：<https://www.jianshu.com/p/a2ce9a117f0a>

大项目-AI仿真器及其他相关项目

- 基本目的：用于生成AI事件的仿真器，给出相应的结果
 - 例如，仿真哈工大校园内的人和车的行为
 - 例如，仿真哈尔滨市的交通情况等
- 随机模拟的基础
 - 事件的随机性都采用概率来进行描述，因此需要对随机变量、随机分布等具有模拟的能力
 - 都是采用模拟的数据来对一些分布进行仿真！
 - 也就是输入一个真实分布，给出一些数据，使得这些数据符合这样的分布
 - 从信息论的角度来看，输入的真实分布是 p ，数据拟合的分布是 q ，这时候计算两者的差别就可以采用相对熵： $D(p||q) = p_i \log \frac{p_i}{q_i}$
 - 当然对于任何机器学习问题，也可以用类似的方式来处理：训练数据 X ，取自某个真实分布 $P(X)$ ，机器学习学习出来一个映射，期望这个映射服从真实分布

概率分布

- 概率分布



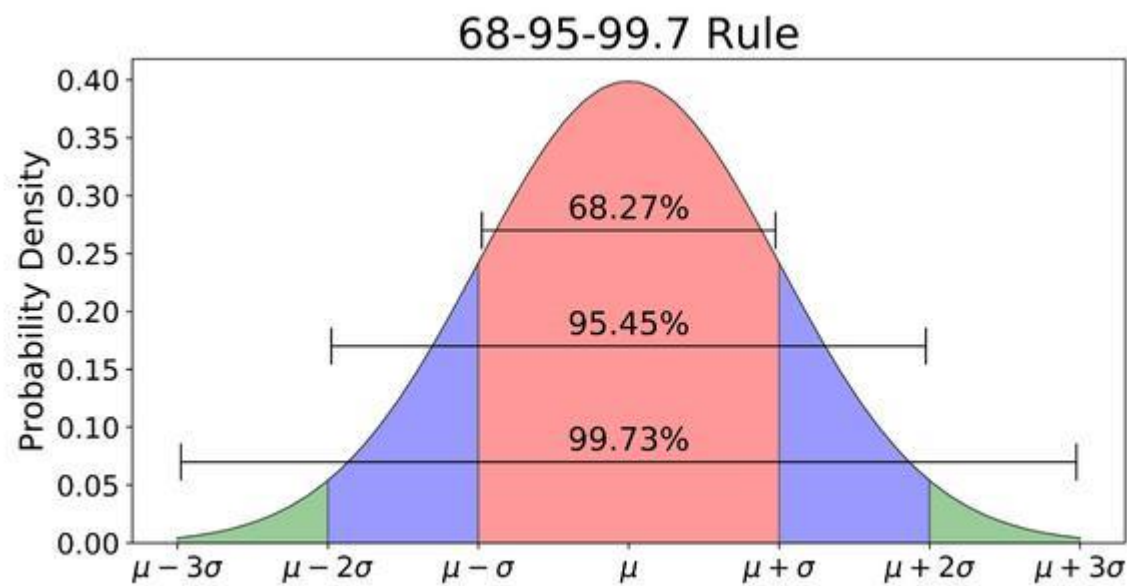
$$y = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

μ = Mean

σ = Standard Deviation

$\pi \approx 3.14159\dots$

$e \approx 2.71828\dots$



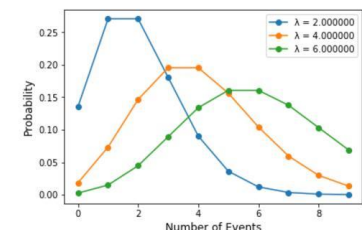
泊松分布(Poisson Distribution)

- 描述单位时间内随机时间发生的次数的概率分布：如某一服务设施在一定时间内的服务请求的次数，电话交换机接到呼叫的次数，汽车站台的侯客人数，机器出现的故障数，自然灾害发生的次数，DNA序列的变异数，放射性原子核的衰变数，激光的分子数分布等

- $$P(X) = \frac{\lambda^x e^{-\lambda}}{X!}$$

- 其中 λ 是表示单位时间（或单位面积）内随机时间的平均发生率，例如预测一天平均有300人来医院就诊，而医院医生的满负荷量是400人，那么出现一天有400人就诊的概率则满足泊松分布

```
for i in range(2, 8, 2):
    n = np.arange(0, 10)
    poisson = stats.poisson.pmf(n, i)
    plt.plot(n, poisson, '-o', label="λ = {}".format(i))
    plt.xlabel('Number of Events', fontsize=12)
    plt.ylabel('Probability', fontsize=12)
    plt.legend()
```



机器学习模型学习联合概率分布($P(X, Y)$)

- 假设所有环境的集合为 ϵ_{all} ,机器学习训练的数据集来自训练环境 ϵ_{train} ,显然 $\epsilon_{train} \subset \epsilon_{all}$, 假设采样得到的训练数据集为: $D_e := \{(x_i^e, y_i^e)\}_{i=1}^{n_e}$, 样本独立同分布, 样本来自某个任意环境 $e \in \epsilon_{train}$.
- 机器学习学习一个预测器 $Y \approx f(X)$, X 为输入数据。这时候有多种情况:
 - $P_{train}(X, Y) = P_{test}(X, Y)$:表示测试集和训练的联合分布一样, 性能好!
 - $P_{train}(X, Y) \neq P_{test}(X, Y)$:表示训练集和测试集的联合分布不一样, 如果 $P_{test}(X, Y)$ 未知, 则模型泛化能力弱
- 联合分布可以分解
 - $P(X, Y) = P(X)P(Y|X)$

机器学习模型学习联合概率分布($P(X, Y)$)

- $P(X, Y) = P(X)P(Y|X)$
- 当出现分布偏差的时候，可进一步根据训练和测试集的分布来进行讨论！
 - 1. 协变量偏移
 - $P_{train}(X) \neq P_{test}(X), P_{train}(Y|X) = P_{test}(Y|X)$
 - 表明训练集和测试集之间的输入变量 X 的分布存在差异，即 $P_{train}(X) \neq P_{test}(X)$ ，但在给定输入 X 的条件下，输出变量 Y 的条件分布在训练集和测试集之间是相等的，即 $P_{train}(Y|X) = P_{test}(Y|X)$

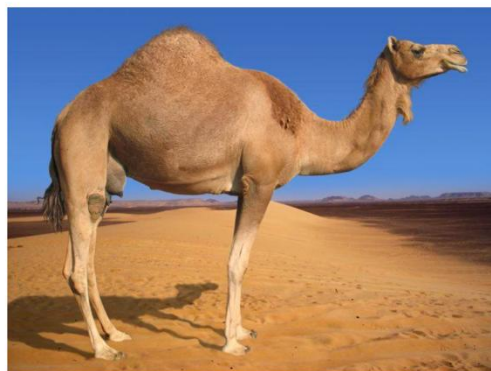
协变量偏移意味着输入的分布发生变化，但输入与输出之间的关系保持不变。现实世界中，这种现象可能源于数据收集过程中的变化或特定环境因素，导致模型需要在不同的输入分布下进行泛化。

机器学习模型学习联合概率分布($P(X, Y)$)

- $P(X, Y) = P(X)P(Y|X)$
- 当出现分布偏差的时候，可进一步根据训练和测试集的分布来进行讨论！
 - 1. 协变量偏移
 - $P_{train}(X) \neq P_{test}(X), P_{train}(Y|X) = P_{test}(Y|X)$
 - 2. 概念偏移
 - $P_{train}(X) = P_{test}(X), P_{train}(Y|X) \neq P_{test}(Y|X)$
 - 概念偏移意味着输入的分布保持不变，但输入与输出之间的关系发生了变化。这可能源于任务的演变、不同领域之间的差异或数据采集过程中的变化。
- 在有监督学习领域，常见第一类协变量偏移！
 - 比如数据增强（data augmentation）、领域迁移（domain adaptation）、元学习（meta learning）等
 - 因果学习（Casual learning）
 - 代表方法为不变风险最小化（invariant risk minimization），简称IRM

因果学习

- 因果学习简单解释



判断图中的动物到底是牛还是骆驼。由于采样的偏差，训练样本中大部分的牛站在绿色的草场上，而骆驼站在棕色的沙漠上。在这样的训练集上用ERM训练完模型之后，研究发现，即使是分类一张清晰的照片——牛站在黄色草地上，如右图，模型也会分类错误。

按照IRM的理论，图片中的背景是 X_S ，即伪相关特征。在训练集中，由于采样偏差，大部分的牛都对应绿色背景，骆驼都对应黄色背景，如果通过ERM来训练模型，模型用背景（伪相关特征）而不用动物本身的特质进行预测，也可以在训练集中达到较低的损失函数值（风险）。然而，这种伪相关特征（spurious feature）会随着环境的变化而变化，即 $P(Y|X_S)$ 会变，与标签 Y 的关系是不稳定的。

机器学习模型

- 学习联合概率分布($P(X, Y)$), 转化为条件分布
 - 采用图模型建模
 - 采用数据驱动来建模
- 采用这种概率分布来研究模型的泛化能力, 以及域迁移能力, 跨模态的理解与分析能力

谢谢！